1. Datenbanksysteme

Disclaimer:

Dieses Material ist lizensiert.

Lizenz: CC BY-NC-ND 4.0 (wenn nicht anders angegeben)

- **Attribution (BY)**
- **Non Commercial (NC)**
- **No Derivatives (ND)**
- **No Warranty**
- **No Liability**

Einzelne Beispiele oder seperat geteilte Dokumente könnten einer anderen Lizenz unterliegen.

Autor: © Korobov Viktor, 2026

Netzwerke:

- LinkedIn
- GitHub
- YouTube
- LeetCode
- Steam

WIP:

Dieses Kursmaterial ist ein lebendiges Dokument und wird mit einem KI Assisstenten gewartet.

Letzter Stand: 02.09.26.

Gebt aufmerksam Acht auf das Gesagte!

Unterstützung:

Bei Anmerkung zum Material kontakiert mich gerne unter:

- viktor.korobov@edu.korobov-solutions.de

Unterstützt mich mit Feedback unter:

- Fragebogen

Datenbanken (DB)

Eine Datenbank ist eine strukturierte Sammlung von Informationen, die in digitaler Form dauerhaft gespeichert und systematisch verwaltet wird. Der zentrale Zweck einer Datenbank besteht darin, große Mengen an Daten so zu organisieren, dass ein effizienter und zuverlässiger Zugriff möglich ist, sei es zum Einfügen, Abfragen, Ändern oder Löschen von Daten.

Im Unterschied zu einfachen Dateisystemen verfolgt eine Datenbank stets einen systematischen Ansatz: Sie basiert auf einem bestimmten Datenbankmodell, das die grundlegende Struktur und Organisation der Daten vorgibt. Auf dieser Grundlage wird ein konkretes Datenmodell entwickelt, das festlegt, wie reale Informationen innerhalb dieser Struktur abgebildet und in Beziehung zueinander gesetzt werden.

Typischerweise bestehen Datenbanken aus Datensätzen, die wiederum in Tabellen organisiert sind. Diese Tabellen bestehen aus Zeilen (Einträgen) und Spalten (Feldern). Jeder Eintrag in einer Tabelle beschreibt eine konkrete Entität, beispielsweise eine Kundin oder ein Produkt. Die Tabelle beschreiben somit einen Entitätstyp, wobei die Spalten die Attribute des Typs sind.

Datenbankmodelle

Ein Datenbankmodell (auch: Datenbankstrukturmodell) ist ein formaler Rahmen, der vorgibt, wie Daten innerhalb eines Datenbanksystems organisiert, gespeichert und miteinander verknüpft werden. Es bildet die theoretische Grundlage, auf der ein konkretes Datenmodell implementiert wird, und bestimmt, welche Strukturprinzipien und Zugriffsmechanismen das DBMS unterstützt.

> **Info:**
>
> DBMS werden im Folgenden erklärt.

Ein Datenbankmodell definiert:

- Wie Entitäten und Beziehungen dargestellt werden (z. B. Tabellen, Dokumente, Knoten)
- Welche Abfragesprachen und Datenoperationen zulässig sind
- Wie Daten intern organisiert und effizient zugänglich gemacht werden

Es beeinflusst maßgeblich die Leistungsfähigkeit, Flexibilität und Einsatzbereiche einer Datenbanklösung.

Beispiele (Datenbankmodelle):

- Relationales Modell
  - Daten werden in Tabellen /(Relationen) gespeichert
  - Jede Tabelle beschreibt einen Entitätstyp, jede Zeile eine Instanz
  - Beziehungen werden über Primär- und Fremdschlüssel realisiert
  - Grundlage für SQL-basierte DBMS
- Dokumentenbasiertes Modell
  - Daten werden als strukturierte Dokumente (meist JSON) gespeichert
- Key-Value-Modell
  - Daten bestehen aus Schlüssel-Wert-Paaren
- Graphdatenmodell
  - Daten werden als Knoten (Entitäten) und Kanten (Beziehungen) gespeichert
- Objektdatenmodell
  - Integration objektorientierter Prinzipien (z. B. Klassen, Vererbung)

> **Info:**
>
> Im Rahmen der Veranstalungen befassen wir uns ausschließlich mit relationalen Datenbanksystemen.

> **Info:**
>
> SQL wird im Folgenden erklärt.

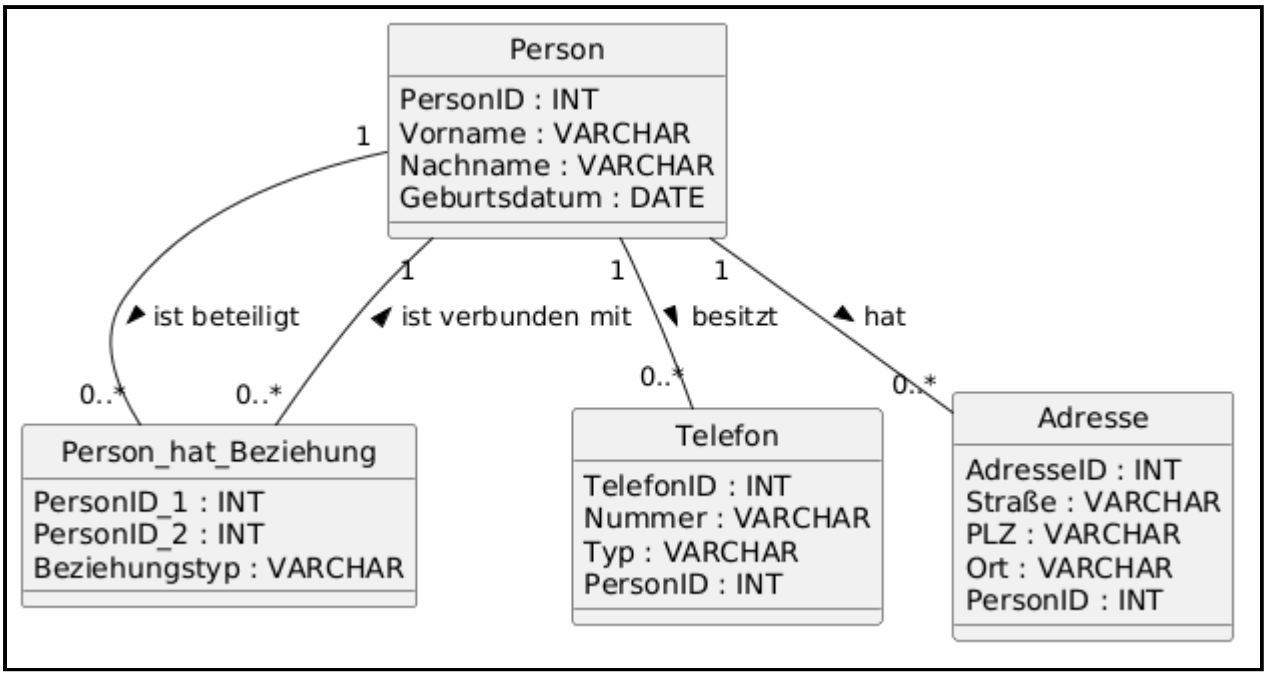
No-SQL-Datenbanken

NoSQL-Datenbanken sind nicht durch eine einheitliche Architektur definiert, sondern durch den Verzicht auf das relationale Modell und die Nutzung eines alternativen Datenbankmodells (Dokumentenbasiert, Key-Value, etc.).

> **Tipp:**
>
> NoSQL-Datenbanken sind nicht zwingend Ersatz, sondern können Ergänzung zu relationalen Systemen – je nach Anwendungsfall - sein.
> In modernen Anwendungen werden können beide Typen kombiniert (polyglotte Persistenz).

Konkrete Datenmodelle

Ein Datenmodell ist eine abstrakte Beschreibung der Struktur von Daten sowie ihrer Beziehungen untereinander. Es dient dazu, reale Informationsstrukturen in ein systematisch erfassbares Schema zu überführen, das dann in einer Datenbank gespeichert und verarbeitet werden kann.

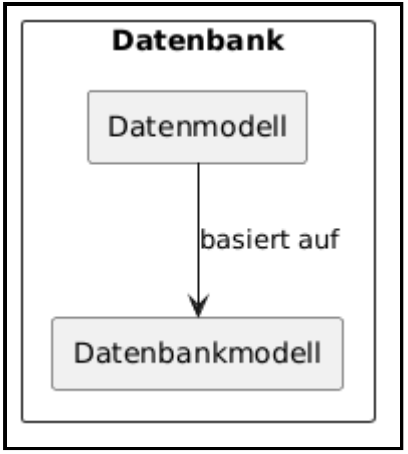


Abgrenzung: Datenmodell vs. Datenbankmodell

Die Begriffe Datenmodell und Datenbankmodell werden im Alltag häufig vermischt, beschreiben jedoch unterschiedliche Konzepte innerhalb der Datenbankentwicklung.

Ein Datenbankmodell ist das abstrakte Ordnungssystem, das ein Datenbanksystem vorgibt. Es definiert die grundsätzliche Strukturierungsform, in der Daten organisiert werden.

Ein Datenmodell hingegen ist die konkrete fachliche Abbildung eines Anwendungsbereichs innerhalb eines gegebenen Datenbankmodells. Es beschreibt, welche Entitäten und Beziehungen in einer bestimmten Domäne existieren.



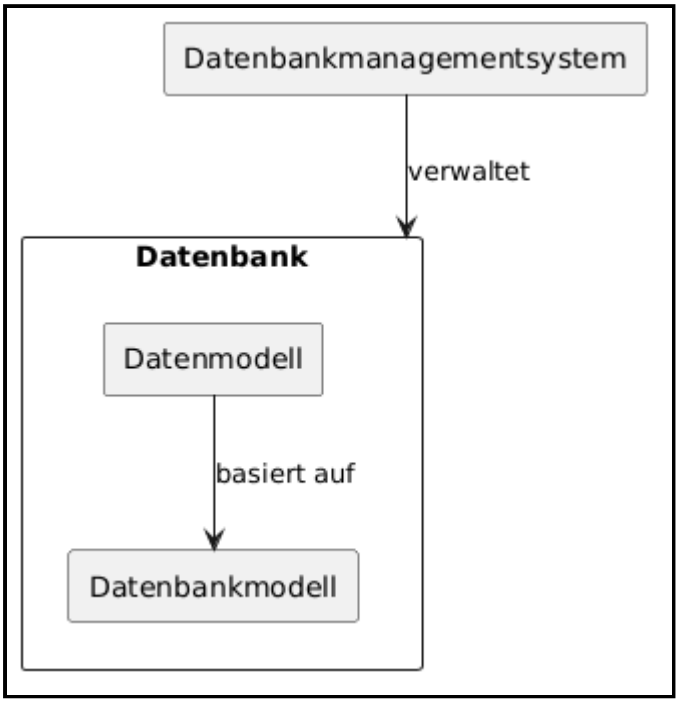
Zusammenfassung:

- Datenbankmodell: Technisches Strukturprinzip, systemweit gültig, vom DBMS vorgegeben
- Datenmodell: Fachlich-inhaltliche Strukturierung, innerhalb einer konkreten Domain

Datenbankmanagementsystem (DBMS)

Ein Datenbankmanagementsystem ist eine Software zur Verwaltung von Datenbanken. Es bildet die Schnittstelle zwischen den Daten (in der Datenbank gespeichert) und den Clients (Anwendungen, Nutzer, etc.), die auf diese Daten zugreifen wollen.

Das DBMS steuert alle Zugriffe auf die Datenbank, regelt, dass Daten korrekt, effizient, sicher und konsistent gespeichert und abgerufen werden, und entlastet Anwendungsprogramme von der direkten Verwaltung der Daten.



Aufgaben eines DBMS:

- Datenorganisation und -speicherung
  - Verwaltung der physischen und logischen Struktur der Daten
  - Unterstützung eines zugrundeliegenden Datenbankmodells

- Datenzugriff und Abfragen
  - Bereitstellung einer Abfragesprache wie SQL
  - Verarbeitung von Anfragen aus Anwendungen
- Mehrbenutzerbetrieb und Transaktionsverwaltung
  - Koordination gleichzeitiger Datenzugriffe
  - Gewährleistung von Konsistenz, Isolation und Dauerhaftigkeit durch Transaktionen (ACID-Prinzipien)
- Zugriffssteuerung
  - Benutzerverwaltung und Rollen
  - Rechtevergabe auf Tabellen- oder Feldebene
- Fehlertoleranz und Datensicherung
  - Sicherungsmechanismen (Backups)
  - Wiederherstellung bei Systemausfällen (Recovery)

Transaktionen

Eine Transaktion ist in der Datenbanktechnik eine logische Folge von Operationen, die als einheitliche, unteilbare Einheit ausgeführt wird. Dabei kann es sich um mehrere Lese-, Schreib-, Änderungs- oder Löschvorgänge handeln, die gemeinsam erfolgreich sein müssen – oder andernfalls vollständig rückgängig gemacht werden.

Transaktionen sind essenziell, um in einem Datenbanksystem um Datenintegrität, Konsistenz und Sicherheit zu gewährleisten – insbesondere bei gleichzeitigen Zugriffen, Systemfehlern oder Abbrüchen während der Verarbeitung.

Das Datenbankmanagementsystem (DBMS) ist die zentrale Instanz zur Verwaltung von Transaktionen.

ACID-Prinzipien

Die ACID-Prinzipien beschreiben die grundlegenden Eigenschaften, die jede Transaktion in einem Datenbanksystem erfüllen muss, um Datenintegrität und Konsistenz zu gewährleisten – auch bei Systemfehlern oder gleichzeitigen Zugriffen. Der Begriff „ACID“ ist ein Akronym für:

- "A" – Atomicity (Atomarität)
  Eine Transaktion wird entweder vollständig oder gar nicht ausgeführt. Wenn ein Teil der Transaktion fehlschlägt, wird die gesamte Transaktion rückgängig gemacht – der Datenbestand bleibt in einem gültigen Zustand.

  > **Beispiel:** Überweist eine Bankanwendung Geld von Konto A auf Konto B, so darf nicht nur das Abbuchen, ohne das anschließende Gutschreiben passieren.

- "C" – Consistency (Konsistenz)
  Eine Transaktion überführt die Daten von einem konsistenten Zustand in einen anderen. Das bedeutet: Alle definierten Regeln, Einschränkungen und Beziehungen (z. B. Fremdschlüssel, Datentypen, Geschäftsregeln) müssen auch nach Durchführung der Transaktion eingehalten bleiben.

  > **Beispiel:** Ein Kunde darf nur zu einer existierenden Adresse zugeordnet werden – dies darf durch keine Transaktion verletzt werden.

- "I" – Isolation (Isolation)
  Gleichzeitig ausgeführte Transaktionen dürfen sich nicht gegenseitig beeinflussen. Für jede Transaktion muss es so erscheinen, als ob sie allein auf dem System ausgeführt wird – unabhängig davon, wie viele andere parallel laufen.

> **Beispiel:** Zwei Bestellungen, die gleichzeitig auf denselben Lagerbestand zugreifen, dürfen keine widersprüchlichen Ergebnisse erzeugen.

- "D" – Durability (Dauerhaftigkeit)
  Sobald eine Transaktion erfolgreich abgeschlossen (committed) wurde, bleiben die Änderungen dauerhaft gespeichert, selbst im Fall eines Stromausfalls oder Systemabsturzes.

  > **Beispiel:** Wenn nach dem Speichern eines neuen Benutzers der Server abstürzt, muss der Benutzer beim Neustart weiterhin vorhanden sein.

> **Tipp:**
>
> ACID ist ein zentrales Qualitätsmerkmal relationaler Datenbanken.
>
> Viele NoSQL-Systeme verzichten jedoch bewusst auf vollständige ACID-Konformität, um Skalierbarkeit und Verfügbarkeit zu verbessern – oft zugunsten sogenannter eventual consistency.

Zusammenfassung der ACID-Prinzipien:

Prinzip	Bedeutung	Ziel
Atomarität	Alles oder nichts	Keine halbfertigen Änderungen
Consistenz	Einhaltung aller Regeln	Keine inkonsistenten Zustände
Isolation	Transaktionen beeinflussen sich nicht	Korrektheit bei Parallelzugriff
Dauerhaftigkeit	Änderungen bleiben bestehen	Verlässliche Datenspeicherung

Structured Query Language (SQL)

SQL (Structured Query Language) ist eine deklarative Abfragesprache, die speziell zur Kommunikation mit relationalen Datenbanksystemen (RDBMS) entwickelt wurde. Sie dient dazu, Daten zu erstellen, abzufragen, zu verändern und zu verwalten.

> **Tipp:**
>
> Eine deklarative Sprache ist eine Programmiersprache, bei der der Fokus auf dem "Was" liegt, also was das gewünschte Ergebnis ist, nicht wie es Schritt für Schritt erreicht werden soll.
>
> Sie beschreibt das Ziel, nicht den genauen Ablauf.

Obwohl SQL eine standardisierte Sprache ist (nach ISO/IEC-Normen), implementieren viele DBMS-Hersteller eigene Erweiterungen oder Varianten, sogenannte SQL-Dialekte. Diese Dialekte folgen im Kern der SQL-Norm, unterscheiden sich aber in:

- Syntaxdetails
- Funktionen und Operatoren
- Daten- und Typdefinitionen
- Systemfunktionen und Prozeduren

Beispiel (SQL-Abfrage):

```sql
SELECT
  p.PersonID,
  p.Vorname,
  p.Nachname,
  p.Geburtsdatum,
  COUNT(b.PersonID_2) AS AnzahlKinder
FROM
  Person p
JOIN
  Person_hat_Beziehung b
  ON p.PersonID = b.PersonID_1
WHERE
  b.Beziehungstyp = 'Mutter'
GROUP BY
  p.PersonID, p.Vorname, p.Nachname, p.Geburtsdatum;
```

- Es wird nach allen Personen gesucht, die in `Person_hat_Beziehung` als `PersonID_1` mit dem Beziehungstyp `"Mutter"` vorkommen.
- Über ein `JOIN` mit `Person` werden die Stammdaten dieser Personen geladen.
- Mit `COUNT` wird gezählt, wie viele Kinder (`PersonID_2`) es pro Mutter gibt.
- Die Gruppierung (`GROUP BY`) stellt sicher, dass pro Mutter genau ein Ergebnis geliefert wird.

Ergebnis (SQL-Abfrage):

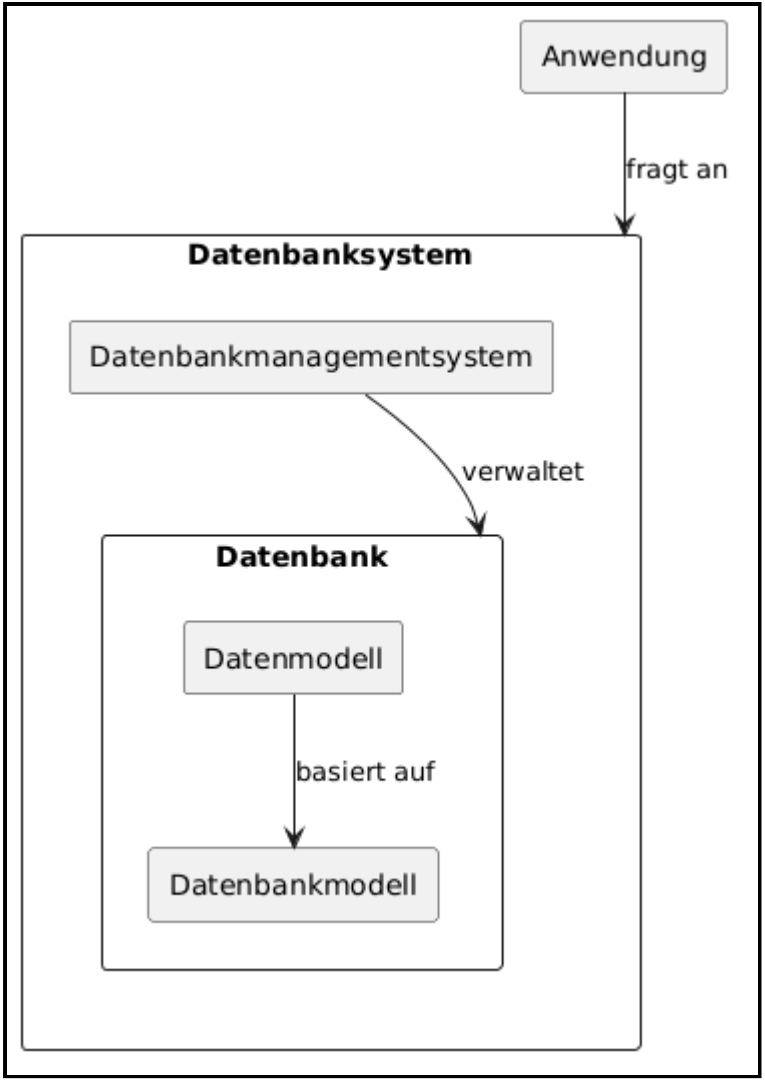
PersonID	Vorname	Nachname	Geburtsdatum	AnzahlKinder
1	Anna	Schmidt	1975-06-12	2
3	Sabine	Maier	1980-03-05	1
7	Claudia	Krüger	1990-09-18	3

Datenbankensysteme (DBS)

Ein Datenbanksystem ist die Gesamtheit aus einer oder mehreren Datenbanken und dem zugehörigen Datenbankmanagementsystem (DBMS). Es stellt eine technische Plattform zur strukturierten Speicherung, Verwaltung und Nutzung von Daten dar und bildet somit das zentrale Rückgrat vieler Softwaresysteme.

Das Datenbanksystem übernimmt alle Aufgaben rund um die Datenhaltung, stellt Schnittstellen zur Datenabfrage und -manipulation bereit und sorgt dafür, dass Daten dauerhaft, konsistent und effizient verfügbar sind – auch bei parallelem Zugriff oder Systemfehlern.

> **Kurz gesagt:**
> Es ist die Kombination aus DBMS und DB



Vorteile des Einsatzes von DBS

Moderne Softwareanwendungen benötigen Mechanismen, um mit Daten zu arbeiten, die über die Laufzeit einzelner Programme hinaus bestehen bleiben. Darüber hinaus ist es vorteilhaft, wenn mit den Daten flexibel und effizient gearbeitet werden kann. Datenbanksysteme erfüllen diese Anforderungen durch:

- Persistenten Datenzugriff
  - Daten bleiben dauerhaft gespeichert, auch nach Beendigung der Anwendung oder bei Stromausfall
  - Ein zentrales Element zur Langzeitdatenhaltung (z. B. Kundendaten, Rechnungen, Protokolle)
- Bequemen Datenzugriff
  - Zugriff über deklarative Sprachen wie SQL oder APIs
  - Unterstützung komplexer Operationen (Filterung, Sortierung, Gruppierung, Joins)
  - Abfragen unabhängig von der konkreten Speicherstruktur
- Effizienten Datenzugriff
  - Interne Optimierung durch Indexe, Caching, Transaktionsverwaltung
  - Unterstützung paralleler Zugriffe durch Sperrmechanismen
  - Ausgelegt auf hohe Datenmengen und Lastverarbeitung

Vielfältigkeit von DBS

Datenbanksystem können sich aus verschiedensten DBMS und DB-Technologien zusammensetzen und somit verschiedenste SQL-Dialekte oder gar völlig andere Abfragesprachen einsetzen.

Jedes Datenbanksystem hat somit seine Vor- und Nachteile und optimale Einsatz-Domain.

Ein Datenbanksystem ist sogar nicht zwangsläufig monolithisch, sondern kann sich aus mehreren Komponenten zusammensetzen, etwa:
- Einzelne oder mehrere DBMS: Kombination relationaler und NoSQL-Systeme innerhalb eines Projekts
- Verschiedene Datenbankmodelle: z. B. Kombination relationaler Tabellen für Geschäftsdaten mit dokumentenbasierten Strukturen für Logs
- Verteilte Architekturen: Systeme, bei denen Daten auf mehrere Knoten oder Standorte verteilt sind
- Spezialisierte Datenbanken für einzelne Anwendungsfälle (z. B. Graphdatenbanken für Netzwerkanalysen)

Dieses Konzept wird auch als polyglotte Persistenz bezeichnet – der Einsatz verschiedener Speichersysteme innerhalb einer Gesamtarchitektur, je nach Anforderung.

Danksagung:

Liebe Lernende, vielen Dank für die Aufmerksamkeit!

Liebe Veranstalter, vielen Dank für die Bühne!

Unterstützung:

Bei Anmerkung zum Material kontakiert mich gerne unter:

- viktor.korobov@edu.korobov-solutions.de

Unterstützt mich mit Feedback unter:

- Fragebogen